

MS&E 228 (Winter 2026) — Lecture 10 Notes

Heterogeneous Treatment Effects

Vasilis Syrgkanis
Stanford University

February 11, 2026

Readings. *Applied Causal Inference Powered by ML and AI*, §14. Relevant papers: Nie & Wager (2021), Foster & Syrgkanis (2023), Kennedy (2023), Athey et al. (2019), Ritzwoller & Syrgkanis (2024), Semenova & Chernozhukov (2021).

Jupyter Notebooks 1. Multi-investment Attribution. 2. Customer Segmentation.

1 Introduction: Beyond Average Effects

This chapter marks a pivotal shift in our study of causal inference, moving beyond the estimation of average treatment effects (ATE) to the richer and more actionable concept of **heterogeneous treatment effects**. While the ATE provides a single, summary measure of a treatment's impact across an entire population, it can mask crucial variations. An intervention might be highly effective for one subgroup, ineffective for another, and even harmful for a third. By averaging these disparate effects, we lose the nuance required for personalized decision-making and risk drawing misleading conclusions.

This chapter delves into the *what*, *why*, and *how* of treatment effect heterogeneity. We will formally define the Conditional Average Treatment Effect (CATE), the central quantity of interest, and explore its importance in applications ranging from personalized medicine to targeted marketing. We will then develop two powerful, machine-learning-based frameworks for CATE estimation: the **DR-Learner** for binary (more broadly categorical) treatments and the **R-Learner** for continuous (and binary) treatments. These methods extend the doubly robust and double machine learning principles we have previously studied, allowing us to leverage the predictive power of flexible ML models while maintaining statistical validity and robustness to nuisance models. Finally, we will address the critical issue of inference, learning how to construct valid confidence intervals for our CATE estimates using both linear models and more complex approaches like Causal Forests, enabling us to quantify the uncertainty in our personalized effect estimates.

2 Why Heterogeneity Matters

Up to this point, our focus has been on estimating a single number to summarize the effect of a treatment, such as the Average Treatment Effect (ATE) or the Average Treatment Effect on the Treated (ATT). While these are valuable for understanding the overall impact of an intervention, they average over the entire population, potentially obscuring important variations. The central question we now turn to is: **how does the treatment effect vary across individuals with different characteristics?**

2.1 The Conditional Average Treatment Effect (CATE)

To formalize the study of treatment effect heterogeneity, we introduce the **Conditional Average Treatment Effect (CATE)**, which is the expected treatment effect for a specific subpopulation defined by a set of observable characteristics, $X = x$.

Definition: Conditional Average Treatment Effect (CATE)

For a **binary treatment** $D \in \{0, 1\}$, the CATE, denoted $\tau(x)$, is defined as the expected difference in potential outcomes for individuals with characteristics $X = x$:

$$\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x]$$

For a **continuous treatment**, the analogous concept is the conditional average marginal effect, which measures the expected local rate of change of the potential outcome with respect to the treatment, for individuals with characteristics $X = x$:

$$\tau(x) = \mathbb{E}\left[\frac{\partial}{\partial d}Y(d) \mid X = x\right]$$

The CATE function, $\tau(x)$, provides a much more granular view than the ATE. While the ATE is the average of the CATE over the entire population, $\text{ATE} = \mathbb{E}[\tau(X)]$, learning the full function $\tau(x)$ allows us to understand for whom a treatment is most and least effective.

2.2 Why CATE Matters

Estimating the CATE is not merely an academic exercise; it has profound practical implications across numerous domains. It allows us to move from one-size-fits-all interventions to personalized, data-driven decision-making.

Enhanced Insights and Hidden Heterogeneity. An intervention might have an average effect that is statistically indistinguishable from zero. However, this null average effect could be the result of strong positive effects for one subgroup canceling out strong negative effects for another. The

ATE would lead us to discard a potentially valuable intervention. By estimating the CATE, we can uncover this hidden heterogeneity.

- **Avoiding Harm:** A new drug could have a very negative effect on a specific, identifiable group of people. An analysis focused solely on the ATE would average this harm over the whole population, potentially missing it entirely and leading to the deployment of a harmful policy. Similarly, an educational reform with a positive average effect might still be detrimental to a specific minority or student subgroup; CATE analysis is crucial for uncovering and preventing such harms.
- **Discovering Hidden Gems (The False Negative Problem):** Conversely, an intervention showing a statistically zero or even slightly negative average effect might be incorrectly dismissed as ineffective. For instance, it could be the case that, for a particular segment of the population identifiable by a specific biomarker, the drug has a substantially large positive effect. An ATE-focused analysis would deprive this group of a beneficial treatment. This motivates using observational data to form hypotheses about which subgroups respond better, which can then be validated with a more targeted Randomized Controlled Trial (RCT) focused specifically on that sub-population.

Personalized Decision Making. The estimation of CATE serves two primary purposes. The first is to gain **advanced insights** into how a population interacts with a treatment. The second is to enable **data-driven, personalized decision-making**, where a CATE model can be used as a black box to optimize choices for individuals without necessarily requiring a deep interpretation of the model’s internal workings. By understanding how the treatment effect varies with individual characteristics, we can tailor interventions to maximize their positive impact and minimize harm.

- **A/B Testing:** In online platforms, it is common for a new feature to have a negligible average effect. A CATE analysis can reveal that some users prefer variant A while others prefer variant B. Instead of rolling out one version to everyone, the platform can implement a personalized policy, showing each user the variant they prefer, leading to an overall improvement in user experience.
- **Healthcare:** The CATE can serve as a predictive biomarker. For instance, a model for $\tau(x)$ could be used to decide which patients should receive a particular drug based on their individual characteristics, leading to more effective and safer treatments.
- **Operations Management:** Companies can use CATE to estimate heterogeneous price elasticities, identifying which customer or product segments are most price-sensitive. This allows for targeted discount strategies that maximize revenue by offering discounts only to those market segments likely to respond by increasing their purchasing behavior.

3 Identification of CATE under Unconfoundedness

Before we can estimate the CATE, we must first establish how to identify it from observable data. In this lecture, we will use the same key assumption that we have been using for average effect: **unconfoundedness**, also known as conditional exogeneity or selection on observables. Under this assumption, the CATE is identified by the **conditional g-formula**.

Identification by Conditioning: Conditional Exogeneity and Overlap

Suppose that the potential outcomes $Y(d)$ are independent of the treatment assignment D conditional on a set of control variables Z , i.e., $Y(d) \perp\!\!\!\perp D \mid Z$. Further, suppose the heterogeneity features X are a subset of the controls, meaning X is measurable with respect to Z . Moreover, the treatment satisfies the overlap assumption, i.e. for all $d \in \{0, 1\}$, we have that $\Pr(D = d \mid Z) \in (0, 1)$, a.s.. Then the CATE is identified by the conditional g -formula:

$$\tau(X) = \mathbb{E}[g_0(1, Z) - g_0(0, Z) \mid X]$$

where $g_0(d, z) = \mathbb{E}[Y \mid D = d, Z = z]$.

This formula tells us that if we can learn the outcome model g_0 , we can identify the CATE by averaging the predicted treatment effect over the distribution of controls Z conditional on X .

Derivation: CATE Identification via G-Formula

We start from the definition of the CATE and apply the law of iterated expectations, then invoke unconfoundedness:

$$\begin{aligned} \tau(X) &:= \mathbb{E}[Y(1) - Y(0) \mid X] \\ &= \mathbb{E}[\mathbb{E}[Y(1) - Y(0) \mid Z] \mid X] \quad (\text{Law of Iterated Exp. \& X is measurable w.r.t. Z}) \\ &= \mathbb{E}[\mathbb{E}[Y(1) \mid Z] - \mathbb{E}[Y(0) \mid Z] \mid X] \quad (\text{Linearity of Expectation}) \\ &= \mathbb{E}[\mathbb{E}[Y(1) \mid D = 1, Z] - \mathbb{E}[Y(0) \mid D = 0, Z] \mid X] \quad (\text{Exog+Overlap}) \\ &= \mathbb{E}[\mathbb{E}[Y \mid D = 1, Z] - \mathbb{E}[Y \mid D = 0, Z] \mid X] \quad (\text{Consistency}) \end{aligned}$$

If we define the outcome model (or g -model) as $g_0(d, z) = \mathbb{E}[Y \mid D = d, Z = z]$, the identification formula follows.

Continuous Treatments

For **continuous treatments**, the same logic applies. The conditional average marginal effect is identified as:

$$\tau(X) := \mathbb{E}\left[\frac{\partial}{\partial d}g_0(D, Z) \mid X\right]$$

4 Approaches to CATE Estimation

Having established the importance and identification of the CATE, we now turn to its estimation. We will review three main strategies: the G-Formula approach (leading to the T-Learner and S-Learner), the Inverse Propensity Weighting (IPW) approach, and finally the Doubly Robust (DR) approach, which combines the strengths of the first two.

4.1 The G-Formula Approach: T-Learner and S-Learner

The identification formula $\tau(X) = \mathbb{E}[g_0(1, Z) - g_0(0, Z) \mid X]$ directly suggests a two-step estimation strategy based on the outcome model, g_0 .

1. **Step 1: Learn the Outcome Model.** Train a machine learning model, $\hat{g}$, to predict the outcome Y based on the treatment D and controls Z .
2. **Step 2: Estimate the CATE.** Use the trained model $\hat{g}$ to predict the potential outcomes for each individual under treatment and control, and then regress this predicted difference $\hat{g}(1, Z_i) - \hat{g}(0, Z_i)$ on the heterogeneity features X_i to learn the CATE function, $\hat{\tau}(X)$.

This general strategy has two common variants, the T-Learner and the S-Learner.

The T-Learner and S-Learner

- The **T-Learner** (for "Two-Learner") fits two separate models for the outcome: one for the treated group $((Y_i, Z_i) \mid D_i = 1)$ to learn $\hat{g}_1(Z) = \mathbb{E}[Y \mid D = 1, Z]$, and another for the control group $((Y_i, Z_i) \mid D_i = 0)$ to learn $\hat{g}_0(Z) = \mathbb{E}[Y \mid D = 0, Z]$. The CATE is then estimated as $\hat{\tau}(X) = \mathbb{E}[\hat{g}_1(Z) - \hat{g}_0(Z) \mid X]$.
- The **S-Learner** (for "Single-Learner") fits a single model for the outcome $\hat{g}(D, Z) = \mathbb{E}[Y \mid D, Z]$, including the treatment indicator D as a feature. The CATE is then estimated as $\hat{\tau}(X) = \mathbb{E}[\hat{g}(1, Z) - \hat{g}(0, Z) \mid X]$.

In the special case where the controls are exactly the heterogeneity features ($Z = X$), the second step of conditioning is unnecessary, and the CATE estimate is simply $\hat{\tau}(X) = \hat{g}_1(X) - \hat{g}_0(X)$ for the T-Learner, and $\hat{\tau}(X) = \hat{g}(1, X) - \hat{g}(0, X)$ for the S-Learner.

While intuitive, these methods can suffer from regularization-induced bias, especially when the treatment effect is small compared to the main effect of the covariates.

Learner vs. Estimator

It is useful to clarify a point of terminology. A method for estimating a single parameter, such as the ATE, is typically called an **estimator**. In contrast, a method for estimating an entire function, such as the CATE, is referred to as a **learner**. A Learner typically produces a model (a function) that can be used to predict a CATE value for any given test point.

4.2 The IPW Approach

An alternative identification strategy uses the propensity score, $p(Z) = P(D = 1 \mid Z)$. We can define the **Inverse Propensity Weighting (IPW)** operator:

$$H_p(D, Z) = \frac{D}{p(Z)} - \frac{1 - D}{1 - p(Z)}$$

This operator has a key property: for any function $g(D, Z)$, it satisfies

$$\mathbb{E}[H_{p_0}(D, Z) \cdot g(D, Z) \mid X] = \mathbb{E}[g(1, Z) - g(0, Z) \mid X], \quad (\text{IPW Property})$$

where p_0 is the true propensity score. We have already seen this property in earlier lectures on average effects, without the conditioning event. It is easy to prove that the conditional version of this property also holds when X is measurable with respect to Z (prove this as an exercise). Applying this property we can prove that the CATE is also identified as a conditional expectation of a weighted outcome:

Identification by Propensity

Under conditional exogeneity and overlap:

$$\tau(X) = \mathbb{E}[H_{p_0}(D, Z) \cdot Y \mid X]$$

Derivation: CATE Identification via Propensities

By the Law of Iterated Expectations and the fact that X is measurable with respect to Z :

$$\begin{aligned} \mathbb{E}[H_{p_0}(D, Z) \cdot Y \mid X] &= \mathbb{E}[\mathbb{E}[H_{p_0}(D, Z) \cdot Y \mid D, Z] \mid X] \\ &= \mathbb{E}[H_{p_0}(D, Z) \cdot \mathbb{E}[Y \mid D, Z] \mid X] \\ &= \mathbb{E}[H_{p_0}(D, Z) \cdot g_0(D, Z) \mid X] \\ &= \mathbb{E}[g_0(1, Z) - g_0(0, Z) \mid X] && (\text{IPW Property}) \\ &= \tau(X) && (\text{g-formula}) \end{aligned}$$

This suggests the **IPW-Learner**: learn an estimate of the propensity score $\hat{p}(Z)$, form the weighted outcomes $H_{\hat{p}}(D, Z) \cdot Y$, and then regress them on X to get $\hat{\tau}(X)$. However, this estimator is known to have very high variance in practice and is rarely used directly.

4.3 The Doubly Robust Approach

The DR approach combines the g-formula and IPW approaches to create an estimator that is robust to misspecification of either the outcome model or the propensity model. The starting point is the key doubly robust identification formula for the CATE:

$$\tau(X) = \mathbb{E}\left[g_0(1, Z) - g_0(0, Z) + H_{p_0}(D, Z) \cdot (Y - g_0(D, Z)) \mid X\right]$$

This is the exact analogue of the doubly-robust formula that we saw for the ATE, with the only difference that we are conditioning on X . It has the same doubly robust properties that we saw for the ATE, i.e., if we define the formula for any pair of models g, p :

$$\tau(X; g, p) = \mathbb{E}\left[g(1, Z) - g(0, Z) + H_p(D, Z) \cdot (Y - g(D, Z)) \mid X\right]$$

Then this formula gives the true ATE if either the outcome model is correct or if the treatment model is correct:

$$\forall p : \tau(X) = \tau(X; g_0, p) \forall g : \tau(X) = \tau(X; g, p_0)$$

Moreover, the error in the formula is related to the product of the two errors (prove as an exercise):

$$\begin{aligned} \tau(X; g, p) - \tau(X) &= \mathbb{E}[(g(D, Z) - g_0(D, Z)) (H_{p_0}(D, Z) - H_p(D, Z)) \mid X] \\ &\lesssim \mathbb{E}[(g(D, Z) - g_0(D, Z)) (p_0(D, Z) - p(D, Z)) \mid X] \end{aligned}$$

The doubly robust formula motivates the definition of the **doubly robust pseudo-outcome**:

Definition: Doubly Robust Pseudo-Outcome

For any outcome model $g(d, z)$ and propensity model $p(z)$, the DR pseudo-outcome is:

$$Y^{DR}(g, p) = g(1, Z) - g(0, Z) + H_p(D, Z) \cdot (Y - g(D, Z))$$

where $H_p(D, Z) = \frac{D}{p(Z)} - \frac{1-D}{1-p(Z)}$. The conditional expectation of the DR pseudo-outcome equals the CATE if either the outcome model or the propensity model is correctly specified, i.e., if $g = g_0$ or $p = p_0$.

$$\mathbb{E}[Y^{DR}(g, p) \mid X] = \tau(X)$$

This theorem means we can estimate the CATE by first estimating the nuisance models $\hat{g}$ and $\hat{p}$, constructing the estimated pseudo-outcomes $Y^{DR}(\hat{g}, \hat{p})$, and then regressing these pseudo-outcomes on the features X . This is the essence of the DR-Learner.

4.4 The DR-Learner Algorithm

To avoid the overfitting bias that arises from using the same data to train the nuisance models and the final CATE model, the DR-Learner algorithm employs cross-fitting, just like the DML and DR estimators for the ATE.

Algorithm: The DR-Learner with Cross-Fitting

The algorithm proceeds in three main stages:

1. **Nuisance Model Training (Cross-Fitted):** Split the data into K folds. For each fold k , train nuisance models on the data outside of fold k :
 - An outcome model $\hat{g}^{(-k)}(d, z)$ to predict Y from D and Z .
 - A propensity score model $\hat{p}^{(-k)}(z)$ to predict D from Z .
2. **Pseudo-Outcome Construction:** For each observation i in fold k , use the nuisance models trained on the other folds to compute the estimated pseudo-outcome,

$$\check{Y}_i^{DR} = \hat{g}^{(-k)}(1, Z_i) - \hat{g}^{(-k)}(0, Z_i) + H_{\hat{p}^{(-k)}}(D_i, Z_i) \cdot (Y_i - \hat{g}^{(-k)}(D_i, Z_i)).$$

3. **Final CATE Model Training:** After cycling through all folds, we have a pseudo-outcome $\check{Y}_i^{DR}$ for every observation i . Train a final regression model, $\hat{\tau}$, to predict these pseudo-outcomes from the covariates X :

$$\hat{\tau}(\cdot) = \arg \min_f \frac{1}{n} \sum_{i=1}^n (\check{Y}_i^{DR} - f(X_i))^2$$

This final model $\hat{\tau}(x)$ is our estimate of the CATE function.

Any regression model can be used for the final stage, from a simple linear model to a complex neural network or gradient boosting machine. The choice of the final model determines the functional form of the estimated CATE.

Choice of Loss Function in Final Stage

The choice of the loss function for this final learner is critical. The squared loss, $\sum (\check{Y}_i^{DR} - f(X_i))^2$, is required because it is the only loss function that recovers the conditional expectation function. Since the CATE is, by definition, the conditional expectation of the DR pseudo-outcome ($\mathbb{E}[Y^{DR} | X = x] = \tau(x)$), using any other loss function would result in estimating a different statistical quantity.

T-Learner and S-Learner Variants of the DR-Learner

The nuisance function $\hat{g}$ can be trained in either the T-Learner way (as two separate models), or in a S-Learner way (as a single model). Both approaches are valid here and give rise to two variants of the DR-Learner algorithm, the DR-Learner(T) and the DR-Learner(S), dependent on which approach is used for the outcome regression.

Python pseudocode for the DR-Learner(S) algorithm.

```
from sklearn.model_selection import KFold
from sklearn.base import clone
import numpy as np

def dr_learner_s(Y, D, X, Z, model_y, model_p, model_final, n_folds=5):
    kf = KFold(n_splits=n_folds, shuffle=True, random_state=42)
    Y_DR = np.zeros(len(Y))

    # 1. K-fold cross-fitting for nuisance models
    for train_idx, test_idx in kf.split(Z):
        # Train nuisance models on out-of-fold data
        g_model = clone(model_y).fit(np.c_[D[train_idx], Z[train_idx]], Y[train_idx])
        p_model = clone(model_p).fit(Z[train_idx], D[train_idx])
        # Predict on in-fold data
        p_hat = p_model.predict_proba(Z[test_idx])[:, 1]
        g_hat_0 = h_model.predict(np.c_[np.zeros(len(test_idx)), Z[test_idx]])
        g_hat_1 = g_model.predict(np.c_[np.ones(len(test_idx)), Z[test_idx]])
        g_hat = (1 - p_hat) * g_hat_0 + p_hat * g_hat_1

        # 2. Compute pseudo-outcomes for the test fold
        Y_DR[test_idx] = (h_hat_1 - h_hat_0
                        + ((D[test_idx] - p_hat) / (p_hat * (1 - p_hat))) * (Y[test_idx] - h_hat))

    # 3. Train final CATE model
    cate_model = clone(model_final).fit(X, Y_DR)
    return cate_model
```

The EconML library provides a `DRLearner` module. To use it, one specifies the ML models for the outcome regression (g_0), the propensity score (p_0), and the final CATE model.

DR-Learner with EconML

```
from econml.dr import DRLearner
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import GradientBoostingClassifier

# Generic DR-Learner with a flexible final model
dr = DRLearner(
    model_regression=GradientBoostingRegressor(),
    model_propensity=GradientBoostingClassifier(),
    model_final=GradientBoostingRegressor()
)
dr.fit(Y, D, X=X, W=W)

# Get CATE predictions
tau_hat = dr.effect(X_test)
```

In this API, the controls Z will be the concatenation of features X and W , i.e. $Z = (X, W)$ and the features X are the features used for heterogeneity.

5 Inference for CATEs

Estimating the CATE function $\hat{\tau}(x)$ is only the first step. Ideally, to use these estimates for decision-making, we must also quantify our uncertainty. That is, we need to construct valid **confidence intervals** for our CATE predictions. This task is challenging because the final CATE model, $\hat{\tau}$, is itself the result of a complex, machine learning estimation process. Generic machine learning models, such as gradient boosting or neural networks, do not typically come with a built-in mechanism for providing valid confidence intervals for their predictions. The distribution of the prediction $\hat{\tau}(x)$ is complex and depends on the entire training dataset in a highly non-linear way. Therefore, we need specialized methods that are designed to provide valid inference in this setting.

The Challenge of Inference with Generic ML Models

Constructing confidence intervals for the predictions of complex, flexible models is a notoriously challenging statistical problem, especially when the covariates are continuous and moderately high-dimensional. Even if there were no first-stage mistakes—that is, even if we knew the true outcome model g_0 and the true propensity model p_0 —it would not be clear how to construct robust confidence intervals for the final regression step. Consequently, when employing a generic machine learning model as the final stage of a DR-Learner, the requirement for confidence intervals is often relaxed. Instead, the focus shifts to ensuring the model has a low mean squared error with respect to the true CATE function. In subsequent lectures, we will explore methods for validating these flexible CATE models on held-out data, including statistical tests to assess whether the model (as a whole) captures a genuine signal related to treatment effect heterogeneity. This is easier than the task of producing confidence intervals for CATE predictions.

We will explore two primary solutions for obtaining valid confidence intervals for CATE estimates:

1. Using a **linear final model** within the DR-Learner framework, which allows us to leverage classical statistical theory for inference.
2. Using a specialized non-parametric model, the **Causal Forest**, which is explicitly designed to produce asymptotically normal predictions with valid standard errors.

5.1 Solution 1: The Linear Final Model and the Best Linear Projection

The most straightforward path to obtaining valid confidence intervals is to restrict the final-stage model in the DR-Learner to be linear in some known features $\phi(X)$. This means we solve the following minimization problem:

$$\min_{\beta} \mathbb{E}_n[(Y^{DR}(\hat{g}, \hat{p}) - \beta' \phi(X))^2]$$

This is a simple multivariate OLS regression of the estimated pseudo-outcomes on the features. The resulting estimator for the CATE is $\hat{\tau}(X) = \hat{\beta}'\phi(X)$. We will refer to this method as the **LinearDRLearner**.

An important question is: what does this linear model estimate if the true CATE function, $\tau(X)$, is not actually linear in $\phi(X)$? The answer is that it estimates the **best linear approximation** (aka **best linear projection**) of the true CATE onto the feature space.

Derivation: Best Linear Projection

Let's analyze the minimizer of the population objective function under the true nuisance functions g_0, p_0 . This is what the DR learner estimator with an OLS final model will be converging too. Let $Y^{DR} := Y^{DR}(g_0, p_0)$. Using the law of iterated expectations and the fact that $\mathbb{E}[Y^{DR} | X] = \tau(X)$, we have:

$$\begin{aligned} \min_{\beta} \mathbb{E}[(Y^{DR} - \beta'\phi(X))^2] &= \min_{\beta} \mathbb{E}[\mathbb{E}[(Y^{DR} - \beta'\phi(X))^2 | X]] \\ &= \min_{\beta} \mathbb{E}[(\mathbb{E}[Y^{DR} | X] - \beta'\phi(X))^2 + \text{Var}(Y^{DR} | X)] \\ &= \min_{\beta} \mathbb{E}[(\tau(X) - \beta'\phi(X))^2] + \mathbb{E}[\text{Var}(Y^{DR} | X)] \end{aligned}$$

Since the second term does not depend on β , the β that minimizes the regression objective is the same β that finds the best linear approximation to the true CATE function $\tau(X)$ in the mean squared error sense. This makes the **LinearDRLearner** a useful tool even when the true CATE is complex.

For this linear DR-Learner, due to the double robustness of the pseudo-outcomes, we can prove an insensitivity and asymptotic normality theorem, analogous to the ATE theorem for the Doubly Robust estimator.

Theorem: Asymptotic Normality of the Linear DR-Learner

Under certain regularity conditions, including consistency of the nuisance models ($\text{RMSE}(\hat{g}) \rightarrow 0$ and $\text{RMSE}(\hat{p}) \rightarrow 0$) and a sufficiently fast convergence rate known as the **product rate condition** ($\sqrt{n} \cdot \text{RMSE}(\hat{g}) \cdot \text{RMSE}(\hat{p}) \rightarrow 0$), the estimated coefficient vector $\hat{\beta}$ is asymptotically normal:

$$\sqrt{n}(\hat{\beta} - \beta_0) \xrightarrow{d} N(0, V)$$

where β_0 is the best linear projection coefficient and V is the asymptotic sandwich variance:

$$V = (\mathbb{E}[\phi(X)\phi(X)'])^{-1} \mathbb{E}[\phi(X)\phi(X)'\epsilon^2] (\mathbb{E}[\phi(X)\phi(X)'])^{-1}$$

with the error term being $\epsilon = Y^{DR} - \beta_0'\phi(X)$.

This theorem is powerful because the asymptotic variance V is exactly what standard statis-

tical packages estimate when computing heteroskedasticity-robust (HC0) standard errors for OLS. Therefore, we can fit a linear model of Y^{DR} on $\phi(X)$, request robust standard errors, and immediately get valid confidence intervals for our coefficients $\hat{\beta}$ and our CATE predictions $\hat{\tau}(x) = \hat{\beta}'\phi(x)$.

$$\sqrt{n}(\hat{\tau}(x) - \tau(x)) = \sqrt{n}\phi(x)'(\hat{\beta} - \beta_0) \xrightarrow{d} N(0, x'Vx)$$

leading to standard error $\mathbf{s.e} = \sqrt{x'\hat{V}x/n}$ and 95% confidence interval:

$$\hat{\tau}(x) \pm 1.96\sqrt{\frac{x'\hat{V}x}{n}} \quad (1)$$

where $\hat{V}$ is the empirical analogue of the sandwich variance.

5.2 Simultaneous Confidence Bands

The previous section showed how to construct a valid (pointwise) 95% confidence interval for $\hat{\tau}(x)$ at a single, pre-specified point x . However, we are often interested in the CATE for many different types of individuals, i.e., for a set of test points $x_1, \dots, x_K$. When we look at many predictions, we run into the **multiple testing** trap. Each individual 95% CI is correct if we had fixed the target point before looking at the data, but if we look at many results and focus on the “interesting” ones, these CIs are not valid. The probability that at least one of the CIs fails to contain the true value can be much larger than 5%.

The Multiple Testing Problem with Pointwise CIs

Pointwise confidence intervals should not be used for exploratory analysis. If one were to analyze 10,000 customers and select the one with the largest apparent effect, the 95% CI for that specific, post-hoc selected customer is not valid. This is a classic multiple testing problem; since each of the 10,000 intervals has a 5% chance of failing, the probability of at least one failure can be very high. For such an analysis, a stronger guarantee is needed.

To address this, we need **simultaneous confidence bands**—a set of intervals $[L_{x_i}, U_{x_i}]$ that are wide enough to ensure joint coverage over all points of interest with a specified probability:

$$\Pr(\forall x \in \{x_1, \dots, x_K\} : \tau(x) \in [L_x, U_x]) \approx 95\%$$

Notice that this guarantee is different from the pointwise confidence interval guarantee:

$$\forall x \in \{x_1, \dots, x_K\} : \Pr(\tau(x) \in [L_x, U_x]) \approx 95\%$$

Moving the “for all” statement inside the probability is crucial for joint coverage and for such exploratory analyses.

Luckily, for linear models, we can construct such intervals with joint coverage by simply “inflating” the constant with which we are multiplying the standard error, e.g., for a 95% joint confidence interval, instead of using $\hat{\tau}(x) \pm 1.95 \cdot \text{s.e.}$, we will use $\hat{\tau}(x) \pm c \cdot \text{s.e.}$ for an appropriate constant c that we need to calculate from the data.

We now describe how to calculate this constant c . For a linear final model, the vector of CATE predictions at a set of test points $X_{test} = \{x_1, \dots, x_K\}^1$ is given by $\hat{\tau}_{test} = X_{test}\hat{\beta}$. Since $\hat{\beta}$ is asymptotically normal, the vector of predictions is approximately jointly Gaussian (if $\log(K) \ll n$):

$$\{\sqrt{n}(\hat{\tau}(x_i) - \tau(x_i))\}_{i=1}^K \stackrel{a}{\sim} N(0, \Sigma), \quad \text{where} \quad \Sigma = X_{test}VX_{test}^\top$$

and V is the sandwich covariance of $\hat{\beta}$. We want to find a critical value c such that the bands $\hat{\tau}(x_i) \pm c \cdot \sqrt{\hat{\Sigma}_{ii}/n}$ have 95% simultaneous coverage. This is equivalent to ensuring that the maximum standardized absolute error is less than c :

$$\Pr\left(\forall i : \tau(x_i) \in \hat{\tau}(x_i) \pm c \cdot \sqrt{\hat{\Sigma}_{ii}/n}\right) = \Pr\left(\max_{i=1, \dots, K} \frac{\sqrt{n}|\hat{\tau}(x_i) - \tau(x_i)|}{\sqrt{\hat{\Sigma}_{ii}}} \leq c\right) \approx 0.95$$

Note that:

$$\left\{ \frac{\sqrt{n}(\hat{\tau}(x_i) - \tau(x_i))}{\sqrt{\hat{\Sigma}_{ii}}} \right\}_{i=1}^K \stackrel{a}{\sim} N(0, D^{-1/2}\Sigma D^{-1/2}),$$

where $D = \text{diag}(\Sigma)$. If we let Z be a random variable distributed as $N(0, D^{-1/2}\Sigma D^{-1/2})$, then:

$$\Pr\left(\max_{i=1, \dots, K} \frac{\sqrt{n}|\hat{\tau}(x_i) - \tau(x_i)|}{\sqrt{\hat{\Sigma}_{ii}}} \leq c\right) \approx \Pr\left(\max_{i=1, \dots, K} |Z_i| \leq c\right)$$

The term inside the probability is the maximum absolute coordinate of a random vector distributed as $N(0, D^{-1/2}\Sigma D^{-1/2})$. We can find the 95th percentile of this quantity by simulation. This will determine the constant c that we need to choose so that we get 95% joint coverage. For such a c :

$$\Pr\left(\max_{i=1, \dots, K} |Z_i| \leq c\right) = 0.95$$

Because c is derived from the maximum of many correlated variables, it will be larger than the 1.96 used for pointwise intervals, resulting in wider, more conservative bands that provide valid joint coverage.

All of these arguments, lead to the following procedure for producing joint confidence intervals.

¹ X_{test} is a matrix where each row i contains the vector x_i .

Algorithm: Constructing Simultaneous Confidence Bands

1. Fit the final linear model $\hat{\beta}$ by regressing Y^{DR} on $\phi(X)$ and compute the HC0 covariance matrix $\hat{V}$ for $\hat{\beta}$.

2. For the test points X_{test} , compute the joint covariance matrix of the predictions:

$$\hat{\Sigma} = X_{test} \hat{V} X_{test}^{\top}.$$

3. Extract the standard errors $\hat{se}_i = \sqrt{\hat{\Sigma}_{ii}}$ and form the correlation matrix $\hat{R} = D^{-1/2} \hat{\Sigma} D^{-1/2}$, where $D = \text{diag}(\hat{\Sigma})$.

4. Simulate large number of draws B from multivariate normal distribution $Z \sim N(0, \hat{R})$.

5. For each draw z_b , compute the maximum absolute coordinate $m_b = \max_i |z_{b,i}|$.

6. The critical value c is the 95th percentile of the sample of maxima, $m_1, \dots, m_B$.

7. The simultaneous confidence bands are $\hat{\tau}(x_i) \pm c \cdot \hat{se}_i$.

Python Pseudocode: Simultaneous Confidence Bands

```
import statsmodels.api as sm
import numpy as np

# 1. Fit final linear model on DR pseudo-outcomes
final_model = sm.OLS(Y_dr, X_final).fit(cov_type='HC0')

# 2. Predictions & joint covariance at test points
preds = final_model.predict(X_test)
V = final_model.cov_params() # HC0 covariance of beta
Sigma = X_test @ V @ X_test.T # joint covariance of tau_hat

# 3. Standardize to correlation matrix
std_devs = np.sqrt(np.diag(Sigma))
D_inv = np.diag(1 / std_devs)
corr = D_inv @ Sigma @ D_inv

# 4. Simulate to find the critical value c
samples = np.random.multivariate_normal(
    np.zeros(len(X_test)), corr, size=5000)
max_abs = np.max(np.abs(samples), axis=1)
c = np.percentile(max_abs, 95)

# 5. Simultaneous confidence bands
lower = preds - c * std_devs
upper = preds + c * std_devs
```

5.3 Solution 2: Causal Forests

What if we believe the CATE function is highly non-linear and cannot be well-approximated by a linear model? The **Causal Forest** provides a flexible, non-parametric alternative that still allows for valid inference.

A Causal Forest is a type of random forest specifically adapted for causal inference. When used as the final model in a DR-Learner (an approach called a **Doubly Robust Causal Forest**), it learns a forest to predict the pseudo-outcomes Y^{DR} from the covariates X . However, it does so with three crucial modifications to the standard random forest algorithm:

1. **Sub-sampling without replacement:** Each tree in the forest is built on a random sub-sample of the data, drawn without replacement and with size $s \ll n$.
2. **Honesty:** For each tree, the data sub-sample is split in two. The first half is used to determine the tree's split structure (the decision rules at each node). The second, separate half is then used to populate the leaves with predictions. This separation prevents the same data from being used for both model selection and estimation, which is a key source of bias.
3. **Balanced splits:** The splitting criterion is modified to require splits that result in a constant fraction of samples in each child node.
4. **Small probability of random splits:** With some small probability a random feature is chosen to split on at each node.

Under these conditions, the predictions of a Causal Forest are asymptotically normal, and valid (pointwise) confidence intervals can be constructed. These are implemented in packages like **grf** in R and in the **grf** module of the **EconML** library in Python.

```
from econml.dr import ForestDRLearner

# DR-Learner with a Causal Forest final model
cf = ForestDRLearner(
    model_regression=GradientBoostingRegressor(),
    model_propensity=GradientBoostingClassifier()
)
cf.fit(Y, D, X=X, W=Z)

# Point estimates with confidence intervals
tau_hat = cf.effect(X_test)
lb, ub = cf.effect_interval(X_test, alpha=0.05)
```

It is important to note that standard implementations of Causal Forests typically provide pointwise confidence intervals. Constructing simultaneous confidence bands for forest-based models is an active area of research, with recent developments making this possible, though it is not yet a standard feature in most packages.

Hyperparameters for Causal Forests

Unlike standard random forests where key hyperparameters include `max_depth` or `min_samples_leaf`, for causal forests, the most critical hyperparameter is the sub-sample size, s . The recommended approach is to use very deep trees with very small leaves (in theory, as low as one sample per leaf, though 5-10 is a practical minimum). The number of trees should be as large as computationally feasible, typically between 4,000 and 10,000, to ensure the quality of the confidence intervals, which are derived from the random variation across trees.

A subtle but important issue is that hyperparameter tuning (e.g., using `cf.tune()`) technically invalidates the confidence intervals. Tuning selects a sub-sample rate that minimizes mean squared error, but the statistical theorems that guarantee CI validity require a sub-sample rate that is often larger than the MSE-optimal one. While tuning is still recommended from a practical standpoint for better point predictions, one should be aware that the resulting CIs are a useful but not fully robust proxy for uncertainty.

Poll: Choosing a CATE Estimator

You have a binary treatment and want to estimate the CATE with valid confidence intervals. Which approach would you choose?

- A. T-Learner with a random forest
- B. DR-Learner with a neural network final model
- C. LinearDRLearner (OLS final model)
- D. Both C and a ForestDRLearner

Answer: The best answer is D. Both the LinearDRLearner and the ForestDRLearner are designed to provide valid confidence intervals. The T-Learner with a standard random forest (A) and the DR-Learner with a standard neural network (B) do not generally provide valid inference. The choice between C and a ForestDRLearner depends on the trade-off between flexibility and the desire for interpretability or simultaneous confidence bands (which are more straightforward for the linear model).

6 Case Study: DR-Learner for Customer Support Interventions

To see the DR-Learner in action, we consider a scenario where a software company wants to understand the impact of different customer support interventions on customer revenue. The company has two main interventions: providing enhanced **Tech Support** and offering a **Discount**. A customer can receive one, both, or neither of these incentives. The goal is to estimate the CATE

of these interventions based on customer characteristics to personalize the support strategy and maximize return on investment (ROI).

This is a multi-treatment problem. We can encode the four possible treatment combinations (None, Tech Support only, Discount only, Both) as a single categorical treatment variable $T \in \{0, 1, 2, 3\}$. We are interested in how the effect of each of these treatments varies with a key customer characteristic: **Size** (the customer's yearly total revenue).

We use the `LinearDRLearner` from EconML, which implements the DR-Learner with a final linear model. This assumes the CATE for each treatment is a linear function of the features, in this case, just **Size**. The nuisance models for the outcome and propensity scores are flexible Gradient Boosting models (`XGBRegressor` and `XGBClassifier`).

```
# Train EconML model with generic helper models
model = LinearDRLearner(
    model_regression=XGBRegressor(learning_rate=0.1, max_depth=3),
    model_propensity=XGBClassifier(learning_rate=0.1, max_depth=3, objective="multi:softprob"),
    random_state=1,
)
# Specify final stage inference type and fit model
model.fit(Y=Y, T=T, X=X, W=W, inference="statsmodels")

# Get summary for each treatment
for i in range(model._d_t[0]):
    display(model.summary(T=i + 1))
```

6.1 Interpreting the CATE Results

The model estimates an intercept and a coefficient for **Size** for each of the three active treatments relative to the 'None' baseline. The results from the accompanying notebook show that the `LinearDRLearner` successfully recovers the underlying ground-truth effects that were used to generate the synthetic data.

Figure 1 visualizes the estimated intercepts and coefficients. The key interpretations are:

- **Tech Support:** This intervention has a high base effect (an intercept of approximately \$5,000), meaning it provides a substantial revenue boost even for the smallest customers. The effect also increases slightly with customer **Size**.
- **Discount:** This intervention has no significant base effect, but its impact grows more strongly with **Size** than tech support does.
- **Both:** The effect of providing both incentives is approximately the sum of the individual effects, suggesting an additive relationship. It combines the high base effect of tech support with the strong positive slope of the discount.

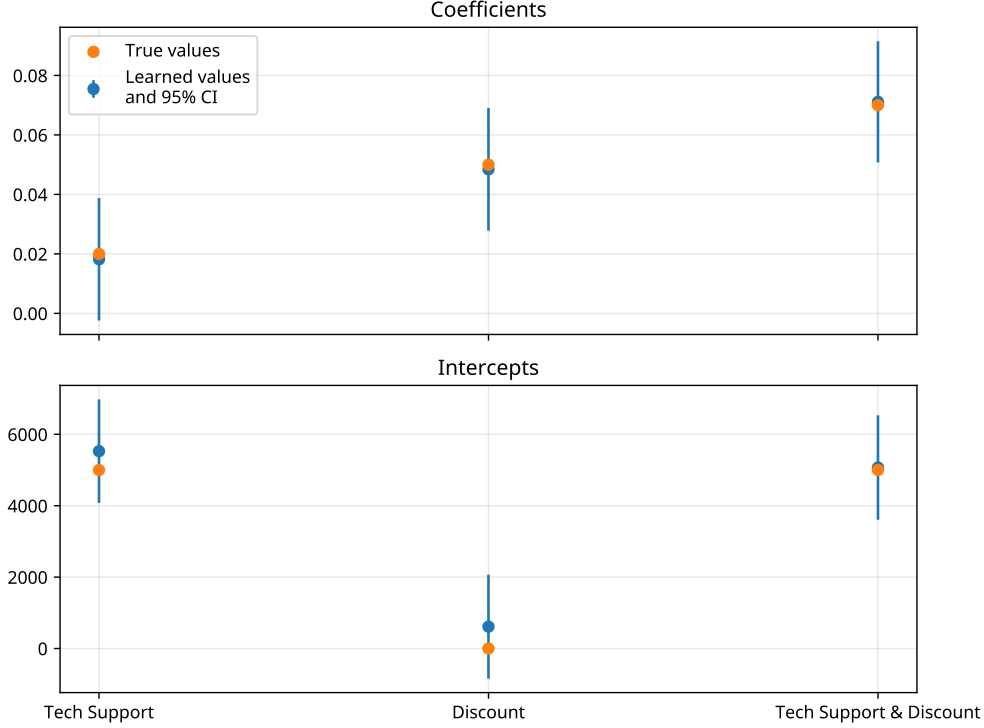


Figure 1: Comparison of learned CATE parameters (coefficients and intercepts) from the **LinearDRLearner** against the true values used in the data generating process. The model was fit on a synthetic dataset of 2,000 customers. The error bars represent 95% confidence intervals. The model accurately recovers the true linear relationship between customer Size and the revenue impact of each intervention.

6.2 From CATE to Optimal Policy

Estimating the CATE is the means to an end. The ultimate goal is to derive an **optimal policy** that assigns the best intervention to each customer to maximize profit. This requires factoring in the cost of each intervention. In this scenario, we assume the cost of Tech Support scales with the customer’s ‘PC Count’ (\$100 per PC), while the Discount has a fixed cost of \$7,000. The expected profit for a given customer and treatment is the estimated CATE (the expected revenue lift) minus the treatment cost.

$$\text{Profit}(x, t) = \hat{\tau}(x, t) - \text{Cost}(x, t)$$

For each customer, we can calculate the expected profit for each of the four possible interventions (including ‘None’, which has zero effect and zero cost) and recommend the one that yields the highest profit.

Figure 2 shows the resulting optimal policy as a function of customer **Size** and ‘PC Count’. The model learns a nuanced set of rules:

- For small customers, it is often best to provide no incentive.
- For small customers with a low number of PCs (making tech support cheap), providing tech

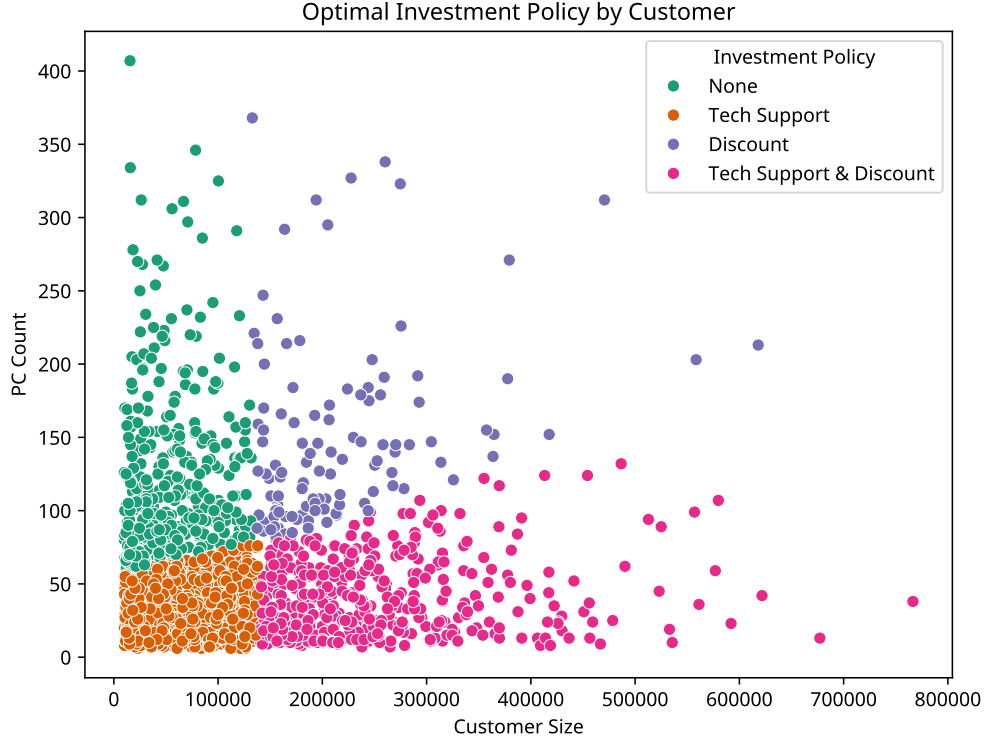


Figure 2: The optimal, profit-maximizing investment policy learned from the CATE estimates. The policy is a function of two customer features: Size and PC Count. The model recommends different interventions for different customer segments to maximize the return on investment. The policy was derived from a **LinearDRLearner** model fit on a synthetic dataset of 2,000 customers, with costs of \$100/PC for tech support and a fixed \$7,000 for a discount.

support is optimal.

- For medium-to-large customers, the discount becomes the most profitable intervention.
- For the largest customers with few PCs, providing both incentives yields the highest return.

This case study demonstrates the full power of the CATE pipeline: moving from observational data to heterogeneous effect estimates, and finally to an interpretable, data-driven, and profitable decision policy.

7 Part IV: The R-Learner for Continuous Treatments

We now turn to the case of **continuous treatments**, such as pricing or dosage. Here, the DR-Learner is not directly applicable. Instead, we develop an analogous estimator based on the principles of Double Machine Learning (DML), which we call the **R-Learner**. The “R” stands for “Robinson,” after the economist Peter Robinson who first proposed a similar residual-on-residual regression for partially linear models² and also stands for “Residual”, since it is a learner based on

²See paper on Root-N-Consistent Semiparametric Regression.

the residualization principle.

Our starting point is a partially linear model that allows for heterogeneous effects. We assume the outcome Y is related to the continuous treatment D and covariates Z (which include X) as follows:

$$Y = \tau(X)D + f(Z) + \epsilon$$

Here, $\tau(X)$ is the CATE function we wish to estimate, which now represents the conditional average marginal effect of the treatment. The function $f(Z)$ captures the confounding effect of all covariates.

7.1 From DML to the R-Learner

Following the logic of DML, we can eliminate the confounding function $f(Z)$ through residualization. Let $h(Z) = \mathbb{E}[Y | Z]$ and $p(Z) = \mathbb{E}[D | Z]$. Note that $h(Z) = \tau(X)p(Z) + f(Z)$. By subtracting the conditional expectations from our model equation, we get:

$$\begin{aligned} Y - h(Z) &= (\tau(X)D + f(Z) + \epsilon) - (\tau(X)p(Z) + f(Z)) \\ \tilde{Y} &= \tau(X)(D - p(Z)) + \epsilon \\ \tilde{Y} &= \tau(X)\tilde{D} + \epsilon \end{aligned}$$

This gives us a simple linear relationship between the outcome residuals, $\tilde{Y} = Y - \mathbb{E}[Y|Z]$, and the treatment residuals, $\tilde{D} = D - \mathbb{E}[D|Z]$. The coefficient on the treatment residual is precisely the CATE function, $\tau(X)$.

This suggests that we can estimate $\tau(X)$ by solving a regression problem. Specifically, we want to find the function $\tau(\cdot)$ that minimizes the squared error in the residualized equation:

$$\min_{\tau(\cdot)} \mathbb{E}[(\tilde{Y} - \tau(X)\tilde{D})^2]$$

This objective is equivalent to a weighted least squares problem:

$$\min_{\tau(\cdot)} \mathbb{E}\left[\left(\frac{\tilde{Y}}{\tilde{D}} - \tau(X)\right)^2 \tilde{D}^2\right]$$

For the same reason why the Double Machine Learning estimator was insensitive to errors in estimating the residual models h and p , we also expect the R-Learner to maintain such insensitivity and robustness.

The R-Learner Objective

The R-Learner estimates the CATE function $\tau(X)$ by performing a **sample-weighted regression** of the ratio of the residuals, $\tilde{Y}/\tilde{D}$, on the covariates X , using the squared treatment residuals, $\tilde{D}^2$, as sample weights.

This weighted regression has an intuitive interpretation. It gives more weight to observations where the treatment residual $\tilde{D}$ is large in magnitude. These are the observations where the treatment assignment has a larger component of random variation after controlling for covariates, providing more information for identifying the causal effect.

7.2 The R-Learner Algorithm

Just like the DR-Learner and DML, the R-Learner algorithm uses cross-fitting to avoid overfitting bias when estimating the nuisance models for the residuals.

Algorithm: The R-Learner with Cross-Fitting

1. **Nuisance Model Training (Cross-Fitted):** Split the data into K folds. For each fold k , train nuisance models on the data outside of fold k :
 - An outcome model $\hat{h}^{(-k)}(Z)$ to predict Y from Z .
 - A treatment model $\hat{p}^{(-k)}(Z)$ to predict D from Z .
2. **Residual Construction:** For each observation i in fold k , compute the estimated residuals:

$$\begin{aligned}\check{Y}_i &= Y_i - \hat{h}^{(-k)}(Z_i) \\ \check{D}_i &= D_i - \hat{p}^{(-k)}(Z_i)\end{aligned}$$

3. **Final CATE Model Training:** After cycling through all folds, train a final weighted regression model, $\hat{\tau}$, to predict the ratio $\check{Y}_i/\check{D}_i$ from X_i , using weights $\check{D}_i^2$.

$$\hat{\tau}(\cdot) = \arg \min_f \sum_{i=1}^n \check{D}_i^2 \left(\frac{\check{Y}_i}{\check{D}_i} - f(X_i) \right)^2$$

As with the DR-Learner, the final model can be anything from a linear model (**LinearDML** in EconML) to a Causal Forest (**CausalForestDML**), providing the same trade-offs between interpretability, inference, and flexibility.

Python Pseudocode: The R-Learner

```
from sklearn.model_selection import KFold
from sklearn.base import clone
import numpy as np

def r_learner(Y, D, X, Z, model_y, model_p, model_final, n_folds=5):

    # Steps 1-3: Cross-fitted residuals (same as DML)
    kf = KFold(n_splits=n_folds, shuffle=True)
    Y_res = np.zeros(n) # outcome residuals
    D_res = np.zeros(n) # treatment residuals
```

```

for train_idx, test_idx in kf.split(X):
    # Train outcome model h(Z) on training fold
    h_model = clone(model_y)
    h_model.fit(Z[train_idx], Y[train_idx])
    Y_res[test_idx] = Y[test_idx] - h_model.predict(Z[test_idx])

    # Train treatment model p(Z) on training fold
    p_model = clone(model_p)
    p_model.fit(Z[train_idx], D[train_idx])
    D_res[test_idx] = D[test_idx] - p_model.predict(Z[test_idx])

# Step 4: Weighted regression for the CATE
cate_model = clone(model_final)
cate_model.fit(X, Y_res / D_res, sample_weight=D_res**2)
return cate_model

cate_model = r_learner(Y, D, X, Z, model_y, model_p, model_final, n_folds=5)
tau_hat = cate_model.predict(X_test)

```

What Does the R-Learner Estimate Under Misspecification?

Note that the partially linear model makes the crucial assumption that the heterogeneous effect only depends on the subset X and not on the whole set of control variables Z . This is a non-trivial assumption. What if the true data generating has the more general form $\mathbb{E}[Y | D, Z] = \theta(Z)D + f(Z)$, i.e., all controls Z can contribute to effect heterogeneity? Note that the latter is without loss of generality of the case when D is binary.

In this case, the R-Learner objective finds the best overlap-weighted approximation to the true CATE function $\theta(Z)$. The derivation is as follows:

$$\begin{aligned}
\min_{\tau} \mathbb{E}[(\tilde{Y} - \tau(X)\tilde{D})^2] &= \min_{\tau} \mathbb{E}[(\mathbb{E}[\tilde{Y} | D, Z] - \tau(X)\tilde{D})^2] \\
&= \min_{\tau} \mathbb{E}[(\theta(Z)\tilde{D} - \tau(X)\tilde{D})^2] \\
&= \min_{\tau} \mathbb{E}[(\theta(Z) - \tau(X))^2 \tilde{D}^2] \\
&= \min_{\tau} \mathbb{E}[(\theta(Z) - \tau(X))^2 \text{Var}(D|Z)]
\end{aligned}$$

This shows the R-Learner minimizes the squared error between the estimated CATE $\tau(X)$ and the true CATE $\theta(Z)$, weighted by the conditional variance of the treatment, $\text{Var}(D|Z)$. This is analogous to how DML for the ATE estimates an overlap-weighted average effect.

DR-Learner vs. R-Learner: A Key Distinction

When the final model is restrictive (e.g., linear), a crucial difference emerges between the DR-Learner and the R-Learner (or DML). The **DR-Learner** estimates the *best linear projection* of the true CATE function. In contrast, the **R-Learner** estimates the *best overlap-weighted linear projection* of the CATE. This distinction diminishes if the final model is highly flexible and can approximate the true CATE well. However, a subtle issue persists: if the features used for the final CATE model, X , are different from the full set of controls, Z , used in the nuisance models, then the R-Learner is computing an overlap-weighted projection of a CATE function that depends on Z , not X , as some function of X . If heterogeneity is driven by control variables in Z that are not included in X , the R-Learner may produce more misleading results than the DR-Learner.

7.3 Generalizations of the R-Learner

The R-Learner framework can be extended to handle more complex treatment structures. What if the treatment effect is not linear in the continuous variable D , but is linear in some features of the treatment, $\phi(D)$? The model becomes:

$$\mathbb{E}[Y \mid D, Z] = \theta(X)' \phi(D) + f(Z)$$

where $\theta(X)$ is now a vector of marginal CATEs, one for each treatment feature in $\phi(D)$. From these marginal CATEs we can construct a heterogeneous dose response curve, as well as a heterogeneous marginal effect curve:

$$\tau(x, d) := \mathbb{E}[\partial_d Y(d) \mid X = x] = \theta(x)' \partial_d \phi(d)$$

The residualization logic extends directly. We residualize both the outcome Y and each treatment feature $\phi_j(D)$ with respect to the controls Z . This yields the relationship:

$$\tilde{Y} = \theta(X)' \tilde{\phi}(D) + \epsilon$$

where $\tilde{\phi}(D) = \phi(D) - \mathbb{E}[\phi(D) \mid Z]$. The final stage is then a multivariate regression of the outcome residuals on the treatment feature residuals, with the coefficients being the marginal CATE functions $\theta(X)$.

Generalized PLM with Heterogeneous Effects

EconML handles this automatically via the `treatment_featurizer` argument, which applies the featurization ϕ to the treatment and then runs the appropriate residual-on-residual regression. The API supports methods for evaluating the conditional marginal effect $\tau(x, d)$ (`marginal_effect(x, d)`) for any test point x , as well as the coefficients $\theta(x)$ (`const_marginal_effect(x)`), as well as the conditional dose response curve $\theta(x)'(\phi(d_1) - \phi(d_0))$ (`effect(x, d0, d1)`).

7.4 The FLAML Custom Metric for the R-Learner

When implementing the final stage of the R-Learner (the weighted regression in Step 4), care must be taken if using automated machine learning tools like FLAML for hyperparameter tuning. The R-Learner objective is to minimize the **un-normalized** weighted sum of squared errors:

$$\mathcal{L}(\tau) = \frac{1}{n} \sum_{i=1}^n \check{D}_i^2 \left(\frac{\check{Y}_i}{\check{D}_i} - \tau(X_i) \right)^2$$

However, many default `sample_weight` implementations in ML libraries—including those used internally by FLAML during cross-validation—**normalize the weights**. That is, they optimize:

$$\mathcal{L}_{\text{normalized}}(\tau) = \frac{\sum_i w_i (y_i - f(X_i))^2}{\sum_i w_i}$$

rather than the raw weighted sum. This normalization is problematic for the R-Learner because the sum of the weights $\sum_i \check{D}_i^2$ can vary across cross-validation folds. If one fold happens to contain observations with larger treatment residuals (and hence larger weights), the normalized metric will undercount the contribution of that fold relative to others. This means the cross-validation score used for hyperparameter selection does not faithfully represent the true R-Learner objective, potentially leading to suboptimal model choices.

The solution is to define a **custom un-normalized metric** for FLAML that computes the raw weighted mean squared error without dividing by the sum of weights. The following code shows how to do this:

Python: Custom FLAML Metric for the R-Learner

```
import numpy as np
from flaml import AutoML

def weighted_sse(
    X_val, y_val, estimator, labels,
    X_train, y_train,
    weight_val=None, weight_train=None,
    *args):
    """Un-normalized weighted MSE on the validation set."""
    y_pred = estimator.predict(X_val)
```

```

    if weight_val is not None:
        loss = np.mean(weight_val * (y_val - y_pred) ** 2)
    else:
        loss = np.mean((y_val - y_pred) ** 2)
    return loss, {"weighted_sse": loss}

automl = AutoML(
    task="regression",
    eval_method='cv',
    metric=weighted_sse, # custom un-normalized metric
    time_budget=60
)
automl.fit(
    X_train=X,
    y_train=Y_res / D_res,
    sample_weight=D_res**2
)

```

The key difference from the default behavior is in the `weighted_sse` function: it computes `np.mean(weight_val * (y_val - y_pred) ** 2)`, which is the average of the weighted squared errors *without* dividing by the sum of the weights. By passing this function as the `metric` argument to FLAML, we ensure that the hyperparameter search optimizes the correct R-Learner objective across all cross-validation folds. This is a subtle but important implementation detail that can affect the quality of the final CATE model.

8 Case Study: R-Learner for Heterogeneous Price Elasticities

We now apply the R-Learner framework to a classic problem in economics and marketing: estimating heterogeneous price elasticities. An online media company wants to understand how different customers respond to price changes for its products. Specifically, they want to know for which customers they should offer a price discount to improve total revenue. This requires estimating the price elasticity of demand, and understanding how it varies with customer characteristics, such as income.

This is a continuous treatment problem, where the treatment D is the log of the price and the outcome Y is the log of the demand (quantity sold). The CATE, $\tau(x)$, is the price elasticity for a customer with characteristics x . The company wants to offer a discount to segments with elastic demand ($\tau(x) < -1$), as a price decrease will lead to a proportionally larger increase in demand, boosting revenue. Conversely, they should not offer a discount to segments with inelastic demand ($\tau(x) > -1$).

We use a synthetic dataset of 10,000 customers. The key feature determining heterogeneity is customer `income`.

8.1 Comparing Parametric and Non-Parametric CATE Models

We fit two different R-Learner models to estimate the price elasticity as a function of income, both implemented in the EconML library:

1. **LinearDML**: This model assumes the CATE, $\tau(x)$, is a parametric (in this case, quadratic) function of income. It uses a **PolynomialFeatures** featurizer to create the basis functions for the final linear model.
2. **CausalForestDML**: This model is non-parametric and uses a Causal Forest for the final stage, allowing it to capture a much more flexible relationship between **income** and price elasticity.

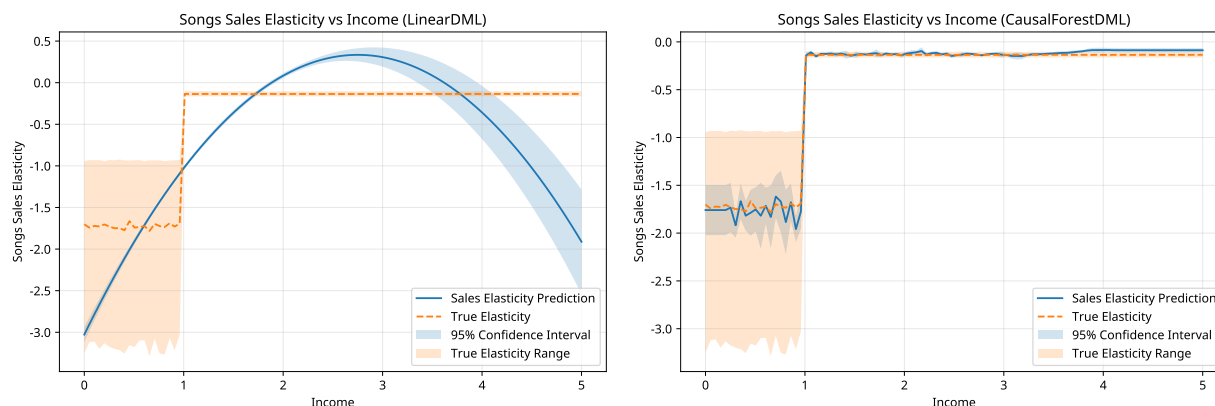


Figure 3: Comparison of CATE estimates for price elasticity from **LinearDML** (left) and **CausalForestDML** (right). The models were fit on a synthetic dataset of 10,000 customers where the true elasticity has a sharp drop at an income level of 1. The **LinearDML** model, constrained to a quadratic form, misses this sharp non-linearity, while the flexible **CausalForestDML** captures it well.

Figure 3 compares the fits from the two models. The true elasticity function is a step function, with a sharp drop at an income level of 1.

- The **LinearDML** model imposes a smooth, quadratic structure on the CATE. While it correctly captures the general trend that elasticity becomes less negative (more inelastic) as income increases, it completely misses the sharp, discontinuous drop. This is a classic example of model misspecification bias.
- The **CausalForestDML**, in contrast, is flexible enough to adapt to the sharp non-linearity in the true CATE function. Its estimate closely tracks the step function shape, and its 95% confidence interval correctly covers the true elasticity across the range of incomes.

8.2 Interpretable Policy Learning

While the **CausalForestDML** model provides an accurate, flexible estimate of the CATE, the model itself is a black box. For decision-making, it is often useful to have a simple, interpretable policy.

We can achieve this by using the `SingleTreePolicyInterpreter` from EconML. This tool takes the complex CATE estimates from the causal forest and learns a single, simple decision tree that approximates the optimal policy.

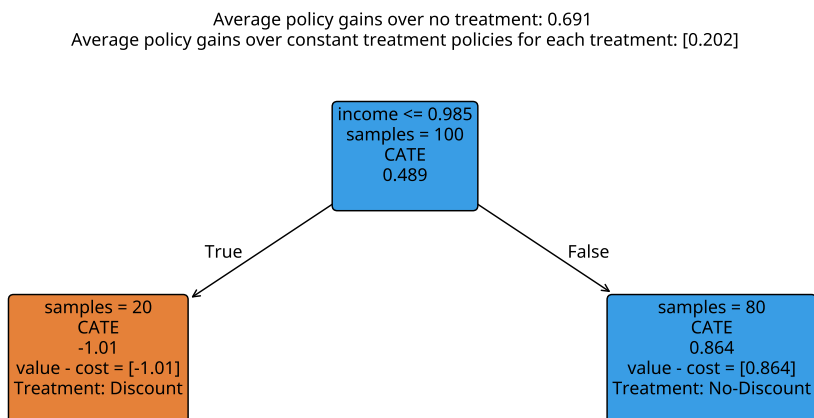


Figure 4: An interpretable policy tree learned from the `CausalForestDML` estimates. The tree provides a simple rule for deciding whether to offer a discount based on customer income to maximize revenue. The data generating process implies that revenue is maximized by offering a discount only when the price elasticity is less than -1 .

Figure 4 shows the learned policy tree. It distills the complex CATE function into a single, actionable rule:

- If `income` ≤ 0.985 : Offer the discount (this is the elastic segment).
- If `income` > 0.985 : Do not offer the discount (this is the inelastic segment).

This simple rule, derived from a sophisticated non-parametric causal model, provides the company with a clear, data-driven strategy for its pricing and promotion efforts, allowing it to easily target discounts to the customers who are most likely to respond, thereby maximizing revenue.

9 Summary: Four Key Takeaways

This chapter provided a comprehensive introduction to the estimation and inference of heterogeneous treatment effects, moving beyond simple population averages to enable personalized, data-driven decision-making.

1. **CATE Unlocks Personalization:** Moving beyond average effects to the Conditional Average Treatment Effect, $\tau(x) = \mathbb{E}[Y(1) - Y(0)|X = x]$, is crucial for personalized policies, enhanced data insights, and discovering hidden heterogeneity where average effects might be misleading.

2. **DR-Learner for Binary Treatments:** The DR-Learner provides a general framework for estimating CATEs for binary treatments. It constructs a doubly robust pseudo-outcome, Y^{DR} , whose conditional expectation is the CATE, turning the causal inference problem into a standard supervised regression task. It can be paired with any ML model for the final CATE estimation.
3. **R-Learner for Continuous Treatments:** The R-Learner extends the principles of Double Machine Learning to estimate heterogeneous effects for continuous treatments. It solves a weighted residual-on-residual regression, effectively finding the best overlap-weighted approximation of the true CATE function, making it robust to regions of low treatment variation.
4. **Inference via Linear Models or Causal Forests:** Obtaining valid confidence intervals for CATEs requires specialized methods. The two primary approaches are using a **linear final model** (e.g., `LinearDRLearner`, `LinearDML`), which provides the best linear projection of the CATE and allows for classical OLS inference, or using a **Causal Forest** (e.g., `ForestDRLearner`, `CausalForestDML`), a non-parametric method specifically designed with honest splitting and sub-sampling to yield asymptotically normal predictions.

Looking Ahead: Model Validation and Selection

This chapter has focused on estimating the CATE and constructing confidence intervals. However, if we do not require confidence intervals and wish to use a highly general, flexible model for the final stage, a key question remains: how can we validate the trained model? The next chapter will explore methods for hyperparameter tuning and model selection for flexible CATE models, as well as statistical tests to assess whether a model has captured a genuine signal related to treatment effect heterogeneity or is merely fitting noise.

10 References

- Nie, X., & Wager, S. (2021). Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika*, 108(2), 299–319.
- Foster, D. J., & Syrgkanis, V. (2023). Orthogonal statistical learning. *The Annals of Statistics*, 51(3), 879–908.
- Kennedy, E. H. (2023). Towards optimal doubly robust estimation of heterogeneous causal effects. *Electronic Journal of Statistics*, 17(2), 3008–3049.
- Athey, S., Tibshirani, J., & Wager, S. (2019). Generalized random forests. *The Annals of Statistics*, 47(2), 1148–1178.
- Ritzwoller, D. M., & Syrgkanis, V. (2024). Simultaneous Inference for Local Structural Parameters with Random Forests. arXiv preprint arXiv:2405.07860.

- Semenova, V., & Chernozhukov, V. (2021). Debiased machine learning of conditional average treatment effects and other causal functions. *The Econometrics Journal*, 24(2), 264-289.
- EconML Documentation: <https://econml.azurewebsites.net/>